

Unit-2

Processes: Process Concept, Process Scheduling, Operations on Processes, Inter-Process Communication.

Threads and Concurrency: Multithreading models, Thread libraries, Threading issues.

CPU Scheduling: Basic concepts, Scheduling criteria, Scheduling algorithms, Multiple processor scheduling.

Part-1

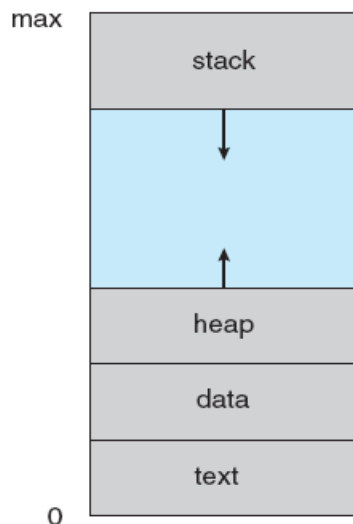
1. Process Concept:

Process:

A process is basically a program in execution. The execution of a process must progress in a sequential fashion.

we write our computer programs in a text file and when we execute this program, it becomes a process which performs all the tasks mentioned in the program.

When a program is loaded into the memory and it becomes a process, it can be divided into four sections stack, heap, text and data. The following image shows a simplified layout of a process inside main memory.



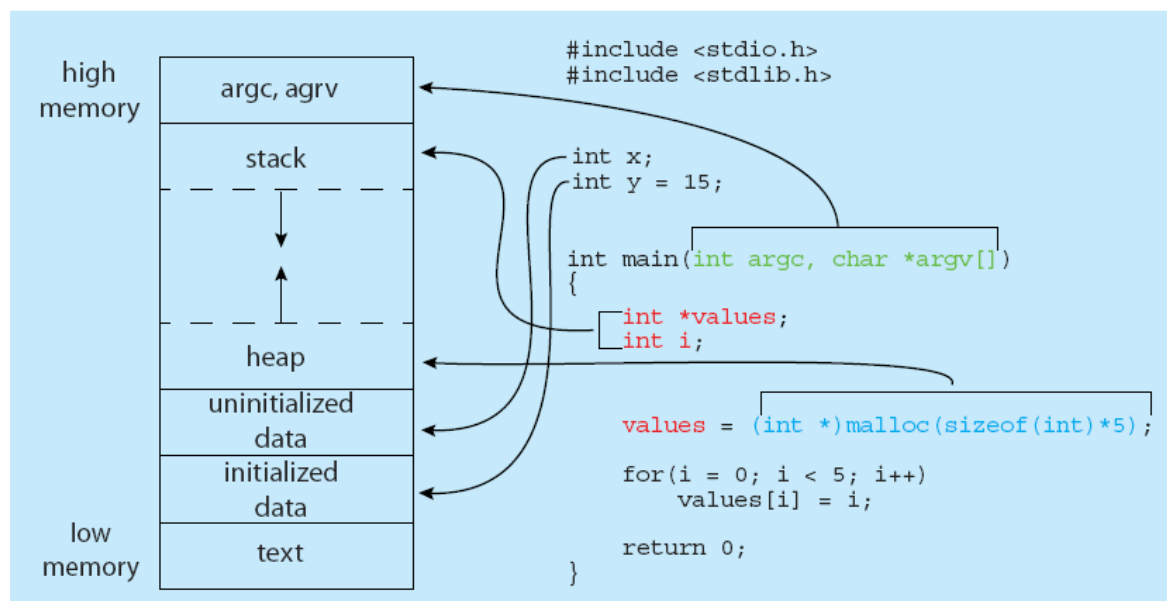
Process in memory.

Stack: The Stack contains the temporary data such as method/function parameters, return address and local variables.

Heap: This is dynamically allocated memory to a process during its run time.

Data: This section contains the global and static variables.

Text: Contains compiled machine code (program instructions).



MEMORY LAYOUT OF A C PROGRAM

Process State:

When a process executes, it passes through different states. These stages may differ in different operating systems. The state of a process is defined in part by the current activity of that process. A process may be in one of the following states:

New: The process is being created.

Ready: The process is waiting to be assigned to a processor.

Running: Instructions are being executed.

Waiting: The process is waiting for some event to occur (such as an I/O completion or reception of a signal).

Terminated: The process has finished execution.

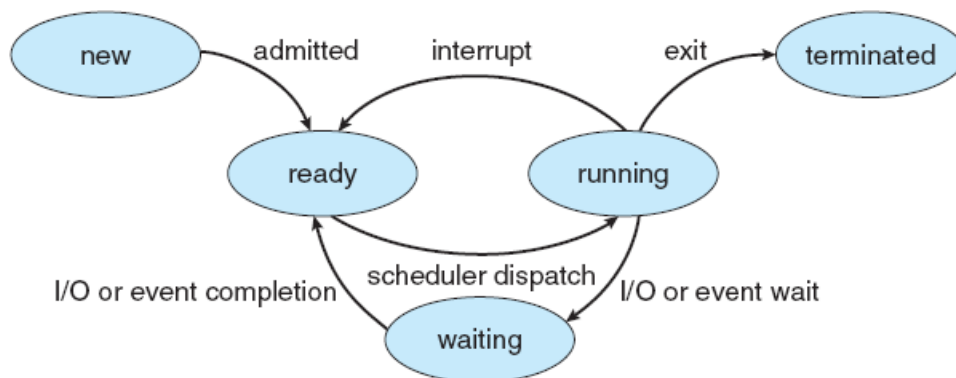
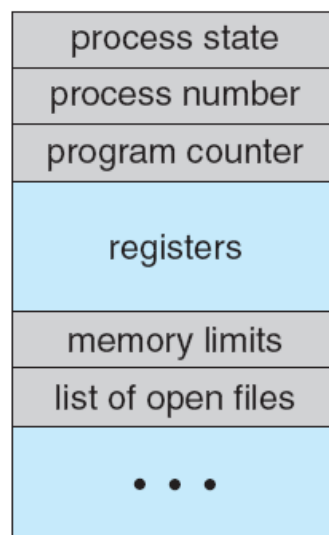


Diagram of process state.

Process Control Block:

A Process Control Block(PCB) is a data structure maintained by the Operating System for every process. The PCB is identified by an integer process ID (PID).It contains many pieces of information associated with a specific process, including these:



Process state: The state may be new, ready, running, waiting, terminated, and so on.

Program counter: The counter indicates the address of the next instruction to be executed for this process.

CPU registers: Various CPU registers where process need to be stored for execution for running state. include accumulators, index registers, stack pointers, and general-purpose registers, plus any condition-code information.

CPU-scheduling information: This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters to schedule the process.

Accounting information: This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.

I/O status information: This information includes the list of I/O devices allocated to the process, a list of open files, and so on.

Thread:

A thread is the unit of execution within a process. A process can have anywhere from just one thread to many threads.

Earlier, the process model assumed that a process executes only one thread at a time.

For example, when a word processor is running, it executes a single sequence of instructions, so it can do only one task at a time. Because of this, the user cannot type text and run the spell checker simultaneously.

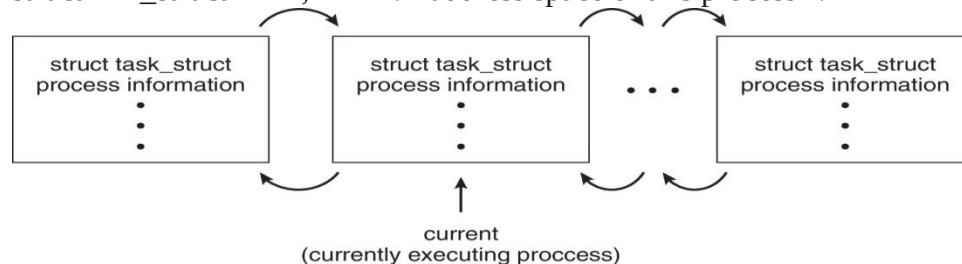
Modern operating systems have improved this model by allowing a process to have multiple threads of execution. This enables a process to perform multiple tasks at the same time. This is especially useful in multicore systems, where different threads can run in parallel.

For instance, in a multithreaded word processor, one thread can handle user input while another thread checks spelling at the same time. To support this, systems that use threads extend the Process Control Block (PCB) to store information about each thread, and other system changes are made to manage threads efficiently.

Process Representation in Linux:

Represented by the C structure `task_struct`

```
pid_t pid;           /* process identifier */
long state;          /* state of the process */
unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm;   /* address space of this process */
```



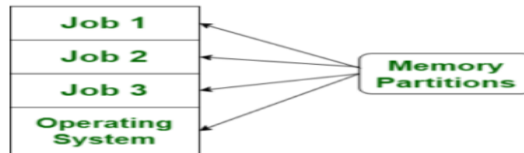
2. Process Scheduling:

Process scheduling is the method used by the operating system to select one process from the ready queue and assign the CPU to it.

How it works (basic idea)

- Processes that are ready to run are placed in a ready queue
- The CPU scheduler selects one process
- The selected process executes on the CPU
- After execution (or interruption), another process is scheduled

Multiprogramming : The **goal of multiprogramming** is to keep the **CPU busy all the time**. This is done by ensuring that whenever one process is waiting (for example, for I/O), another process is ready to use the CPU. This helps **maximize CPU utilization**.



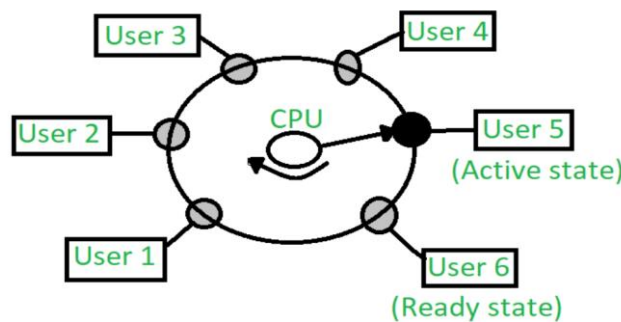
Advantages:

- Multiprogramming systems involve high CPU utilization. Thus, waiting time is reduced.
- Less waiting and response time

Disadvantages:

- Process scheduling is difficult
- Main memory management is required

Time Sharing : The **goal of time sharing** is to allow **multiple users or programs to interact simultaneously**. To achieve this, the CPU rapidly switches between processes so that each program appears to run continuously, even though each one gets the CPU only for a short time.



To achieve both goals, the **process scheduler** selects one process from the available processes and assigns it to a **CPU core** for execution.

Key points:

- **Each CPU core can run only one process at a time.**
- In a **single-core system**, only one process can be running at any moment.
- In a **multicore system**, multiple processes can run at the same time—one on each core.
- If there are **more processes than CPU cores**, the extra processes must **wait** until a core becomes free.

The **degree of multiprogramming** refers to the **number of processes currently loaded in main memory**.

To balance multiprogramming and time sharing effectively, the scheduler considers the **behaviour of processes**:

- An **I/O-bound process** spends most of its time performing input/output operations and waits frequently.
- A **CPU-bound process** spends most of its time doing computations and makes fewer I/O requests.

By mixing I/O-bound and CPU-bound processes, the operating system can keep both the **CPU and I/O devices busy**, improving overall system performance.

Scheduling Queues:

1. Job Queue

- Contains all processes in the system
- Processes are usually stored on disk
- New processes enter the system through this queue

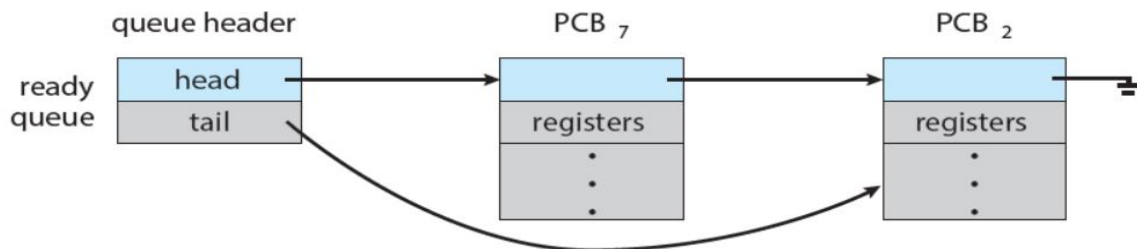
2. Ready Queue

- Contains processes that are in main memory
- These processes are ready to execute
- CPU scheduler selects a process from this queue

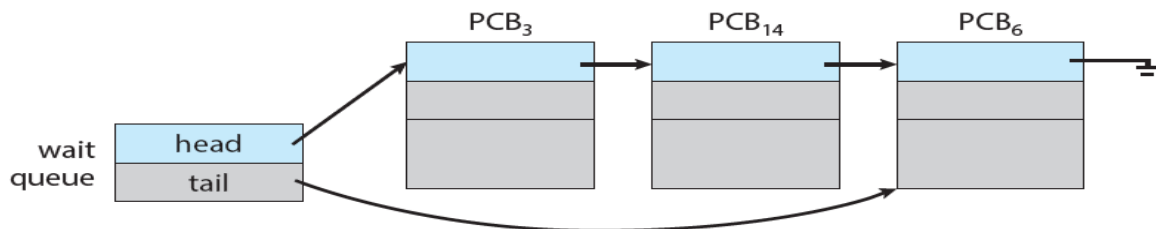
3. Waiting Queue (I/O Queue)

- Contains processes waiting for I/O operations
- Once I/O completes, the process moves back to the ready queue

As processes enter the system, they are put into a ready queue, where they are ready and waiting to execute on a CPU's core. This queue is generally stored as a linked list; a ready-queue header contains pointers to the first PCB in the list, and each PCB includes a pointer field that points to the next PCB in the ready queue.



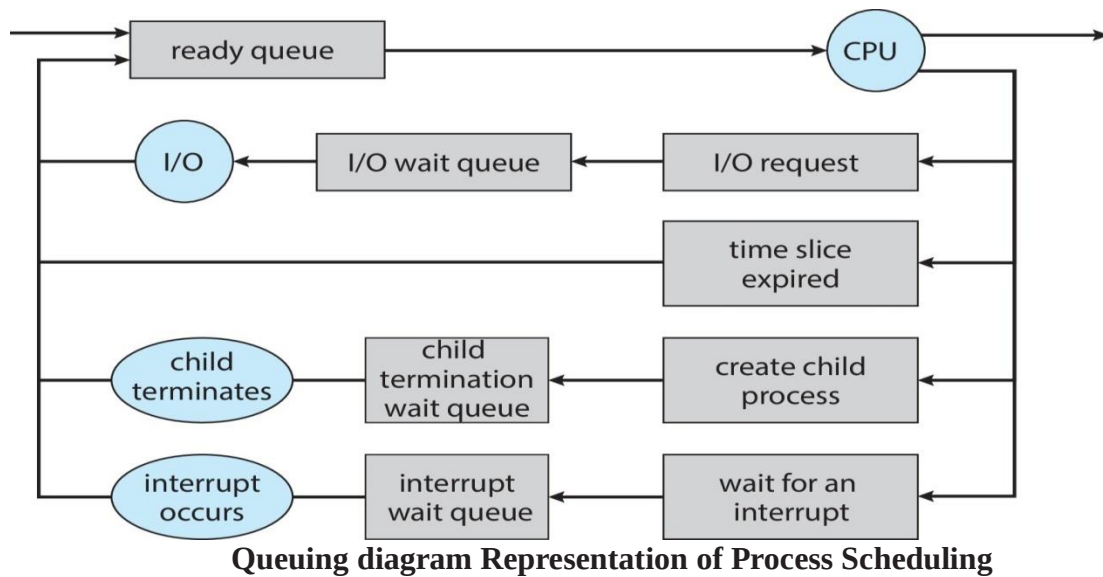
Processes that are waiting for a certain event to occur — such as completion of I/O — are placed in a wait queue.



A common representation of process scheduling is a queuing diagram, such as, Each rectangular box represents a queue. Two types of queues are present: the ready queue and a set of wait queues. The circles represent the resources that serve the queues, and the arrows indicate the flow of processes in the system. A new process is initially put in the ready queue. It waits there until it is selected for execution, or dispatched. Once the process is allocated the CPU and is executing, one of several events could occur:

- The process could issue an I/O request and then be placed in an I/O queue.
- The process could create a new child process and wait for the child's termination.
- The process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the ready queue.

In the first two cases, the process eventually switches from the waiting state to the ready state and is then put back in the ready queue. A process continues this cycle until it terminates, at which time it is removed from all queues and has its PCB and resources deallocated.



CPU Scheduling:

During its lifetime, a process keeps moving between the **ready queue** and different **waiting (I/O) queues**.

The **CPU scheduler** is responsible for choosing one process from the ready queue and giving it a **CPU core** to run.

The scheduler must make decisions **very frequently**:

- An **I/O-bound process** runs only for a short time (a few milliseconds) before it needs input/output and goes to a waiting state.
- A **CPU-bound process** needs the CPU for a longer time, but the scheduler does **not allow it to run continuously** for too long.

To ensure fairness and efficient CPU use, the scheduler may **forcefully take the CPU away** from a running process and allow another process to run.

Sometimes, when **main memory is overused**, the operating system temporarily **removes a process from memory**, saves its current state on disk, and later **brings it back into memory** to continue execution from where it stopped.

This process is called **swapping**.

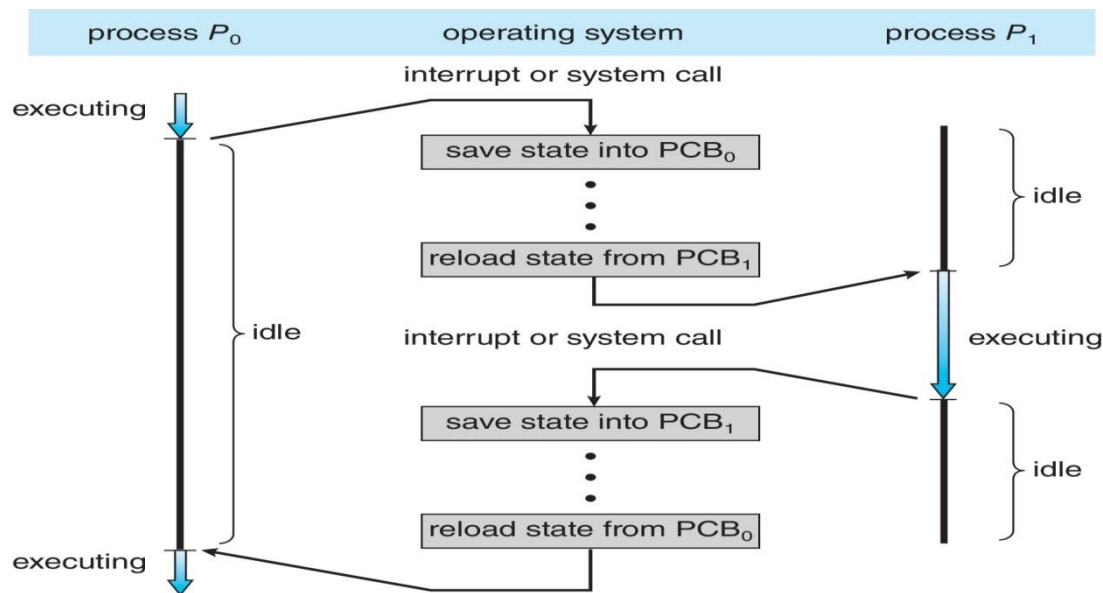
- **Swapping** reduces the number of processes competing for the CPU at the same time.
- It is mainly used when **memory is overcommitted** and space needs to be freed.

Because of these frequent decisions, the CPU scheduler typically runs **at least once every 100 milliseconds**, and often much more frequently.

Context Switch:

- When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process via a context switch.
- Context of a process represented in the PCB
- Context-switch time is overhead; The system does no useful work while switching
- Time dependent on hardware support
- State save- save the current process state
- State reload- resume the process state.

A **context switch** occurs when the CPU switches from one process to another.



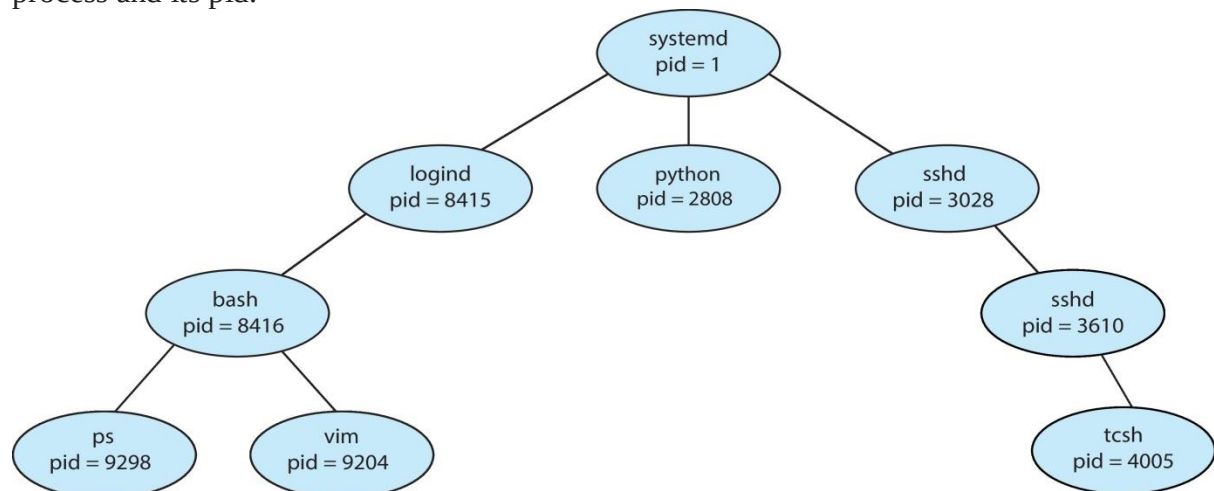
3. Operations on Processes:

The processes in most systems can execute concurrently, and they may be created and deleted dynamically. Thus, these systems must provide a mechanism for process creation and termination.

Process Creation:

A process may create several new processes. The creating process is called a parent process, and the new processes are called the children of that process. Each of these new processes may in turn create other processes, forming a tree of processes. Most operating systems identify processes according to a unique process identifier (or pid), which is typically an integer number. The pid provides a unique value for each process in the system.

Figure shows a typical process tree for the Linux operating system, showing the name of each process and its pid.



Resource sharing options:

- Parent and children share all resources
- Children share subset of parent's resources

Execution options:

- Parent and children execute concurrently
- Parent waits until children terminate

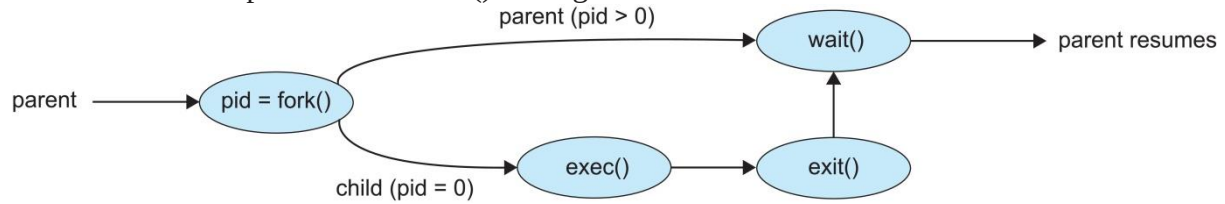
Address space

- Child duplicate of parent
- Child has a program loaded into it

UNIX examples

- `fork()` system call creates new process

- `exec()` system call used after a `fork()` to replace the process' memory space with a new program
- Parent process calls `wait()` waiting for the child to terminate



Process Termination:

- Process executes last statement and then asks the operating system to delete it using the **exit()** system call.
 - Returns status data from child to parent (via **wait()**)
 - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **abort()** system call. Some reasons for doing so:
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - The parent is exiting, and the operating systems does not allow a child to continue if its parent terminates
- Some operating systems do not allow child to exists if its parent has terminated. If a process terminates, then all its children must also be terminated.
 - **cascading termination.** All children, grandchildren, etc., are terminated.
 - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the **wait()** system call. The call returns status information and the pid of the terminated process

pid = wait(&status);

- If no parent waiting (did not invoke **wait()**) process is a **zombie**
- If parent terminated without invoking **wait()**, process is an **orphan**.

Example:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid = fork();
    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    } else if (pid == 0) {
        printf("Child Process: PID = %d\n", getpid());
        execl("/bin/ls", "ls", "-l", (char *)NULL);
        // If execl() is not successful, this line will not be executed
        perror("execl failed");
        exit(1);
    } else {
        // Parent process
    }
}
  
```



```

printf("Parent Process: PID = %d\n", getpid());
wait(NULL); // Wait for child process to complete
printf("Parent Process: Child Completed\n");
}
return 0;
}

```

4. Inter-process Communication:

Processes executing concurrently in the operating system may be either independent processes or cooperating processes. A process is independent if it can't affect or be affected by the other processes executing in the system. A process is cooperating if it can affect or be affected by the other processes executing in the system. Clearly, any process that shares data with other processes is a cooperating process.

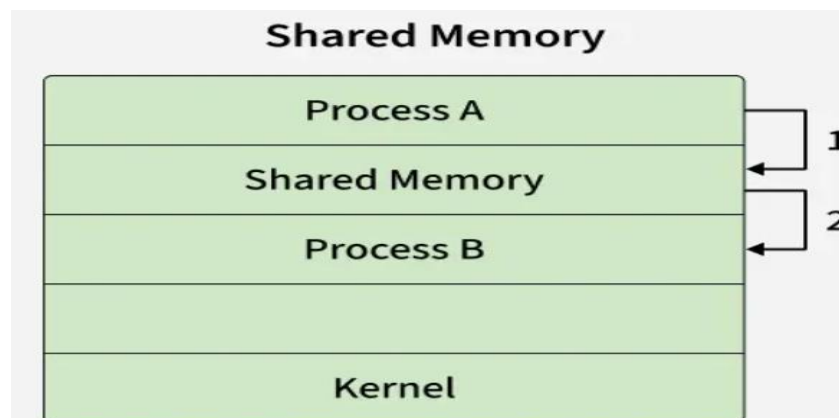
There are several reasons for providing an environment that allows process cooperation:

- **Information sharing:** Since several applications may be interested in the same piece of information (for instance, copying and pasting), we must provide an environment to allow concurrent access to such information.
- **Computation speedup:** If we want a particular task to run faster, we must break it into subtasks, each of which will be executing in parallel with the others. Notice that such a speedup can be achieved only if the computer has multiple processing cores.
- **Modularity:** We may want to construct the system in a modular fashion, dividing the system functions into separate processes or threads.

Cooperating processes require an inter-process communication (IPC) mechanism that will allow them to exchange data—that is, send data to and receive data from each other. There are two fundamental models of inter-process communication: shared memory and message passing.

Shared Memory

Communication between processes using shared memory requires processes to share some variable and it completely depends on how the programmer will implement it. Processes can use shared memory for extracting information as a record from another process as well as for delivering any specific information to other processes.

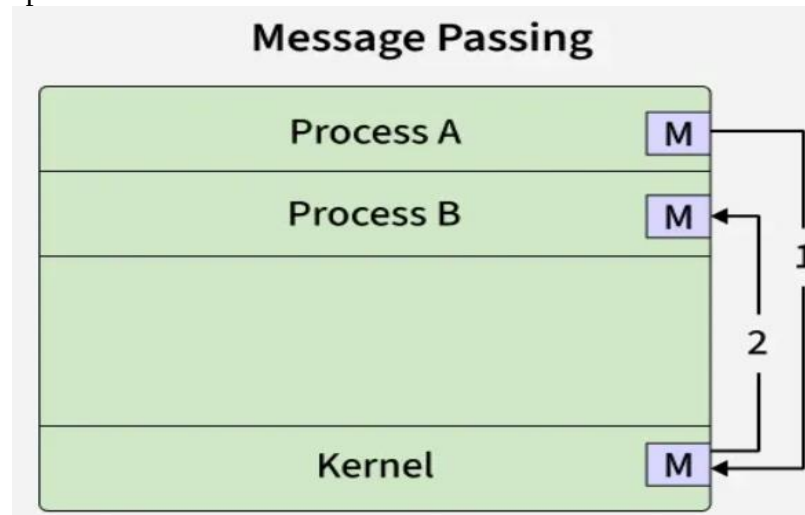


- In the above shared memory model, a common memory space is allocated by the kernel.
- Process A writes data into the shared memory region (Step 1).
- Process B can then directly read this data from the same shared memory region (Step 2).
- Since both processes access the same memory segment, this method is fast but requires synchronization mechanisms (like semaphores) to avoid conflicts when multiple processes read/write simultaneously.

Message Passing

Message Passing is a method where processes communicate by sending and receiving messages to exchange data.

- In this method, one process sends a message and the other process receives it, allowing them to share information.
- Message Passing can be achieved through different methods like Sockets, Message Queues or Pipes.

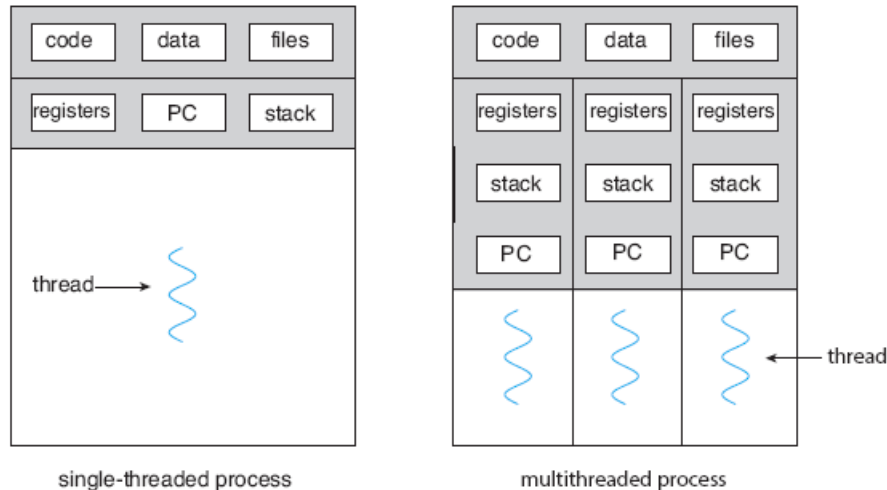


- In the above message passing model, processes exchange information by sending and receiving messages through the kernel.
- Process A sends a message to the kernel (Step 1).
- The kernel then delivers the message to Process B (Step 2).
- Here, processes do not share memory directly. Instead, communication happens via system calls (`send()`, `recv()`, or similar).
- This method is simpler and safer than shared memory because there's no risk of overwriting shared data, but it incurs more overhead due to kernel involvement.

Part-2

1. introduction thread:

A thread is a basic unit of CPU utilization; it comprises a thread ID, a program counter (PC), a register set, and a stack. It shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals. A traditional process has a single thread of control. If a process has multiple threads of control, it can perform more than one task at a time. Figure illustrates the difference between a traditional single-threaded process and a multithreaded process.



Most software applications that run on modern computers and mobile devices are multithreaded. An application typically is implemented as a separate process with several threads of control. Below we highlight a few examples of multithreaded applications:

- An application that creates photo thumbnails from a collection of images may use a separate thread to generate a thumbnail from each separate image.
- A web browser might have one thread display images or text while another thread retrieves data from the network.
- A word processor may have a thread for displaying graphics, another thread for responding to keystrokes from the user, and a third thread for performing spelling and grammar checking in the background.

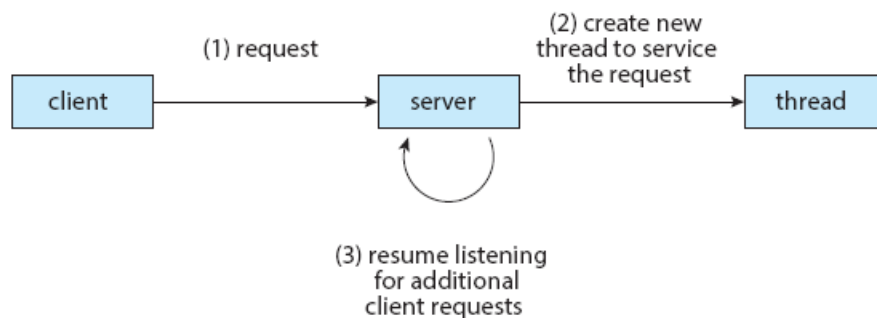


Figure 4.2 Multithreaded server architecture.

A modern web servers use a **multithreaded model**, where a single process contains multiple threads. The server maintains a main thread that listens for incoming client requests. When a request arrives, the server creates a new thread to handle that request while the main thread continues listening for additional requests. Since threads are lightweight and share the same memory space, they are faster to create and consume fewer resources than processes. As a result, multithreaded servers provide better CPU utilization, faster response times, and improved scalability, making them well suited for high-load and multicore systems.

Benefits:

The benefits of multithreaded programming can be broken down into four major categories:

- **Responsiveness:**

Multithreading keeps an application responsive even when one part is busy. Time-consuming tasks can run in separate threads, allowing the user interface to remain active.

- **Resource Sharing:**

Threads share the same memory and resources of a process by default. This makes sharing code and data easier compared to separate processes.

- **Economy:** Allocating memory and resources for process creation is costly.

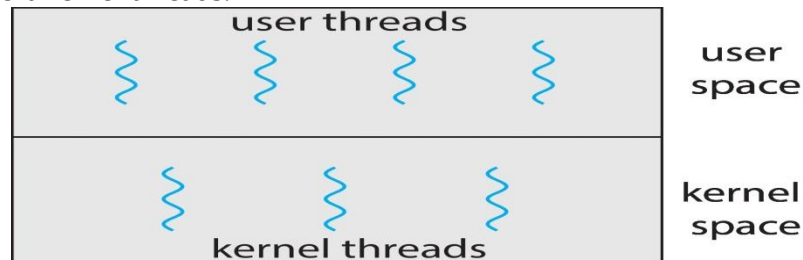
Creating and switching between threads requires less time and memory than processes because threads share resources.

- **Scalability:**

In multiprocessor systems, multiple threads can run in parallel on different cores, improving performance compared to single-threaded programs.

2. Multi-threading Model:

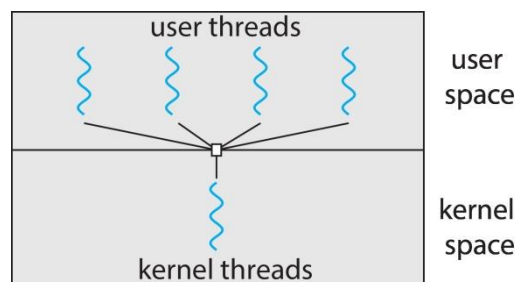
- Threads may be provided either at the user level, for **user threads**, or by the kernel, for **kernel threads**.
- User threads are supported above the kernel and are managed without kernel support, whereas kernel threads are supported and managed directly by the operating system.
- Virtually all contemporary operating systems—including Windows, Linux, and macOS—support kernel threads.



- Ultimately, a relationship must exist between user threads and kernel threads, we look at three common ways of establishing such a relationship.

- ▶ Many-to-One
- ▶ One-to-One
- ▶ Many-to-Many

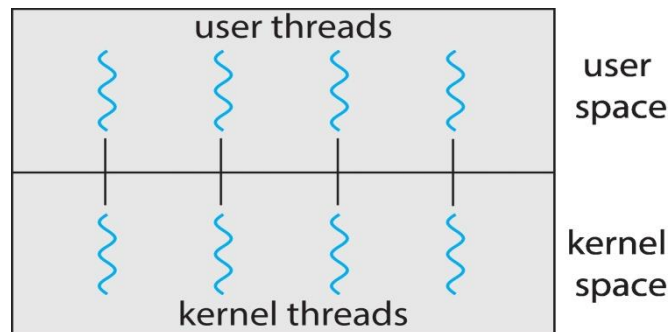
Many-to-One:



- Many user-level threads mapped to single kernel thread.
- One thread blocking causes the entire system to be blocked.

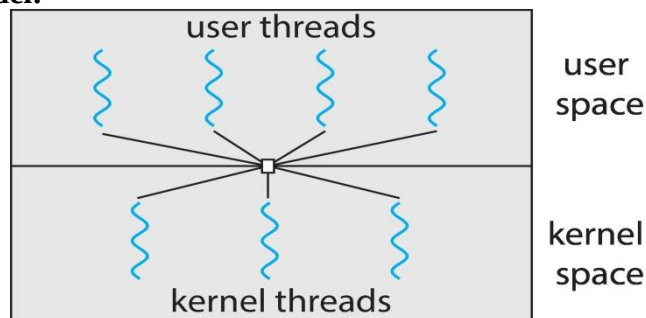
- Multiple threads may not run in parallel on multicore system because only one may be in kernel at a time.
- Few systems currently use this model.
- Examples:
 - Solaris Green Threads
 - GNU Portable Threads

One-to-One:



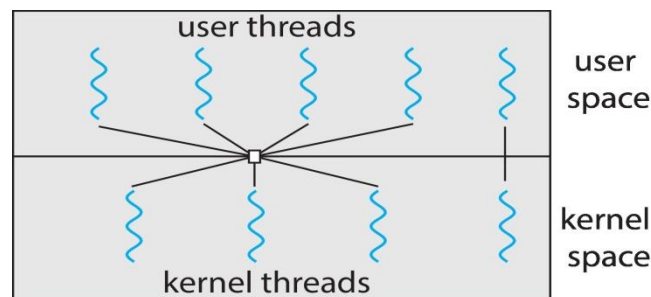
- Each user thread is mapped to exactly one kernel thread
- Provides higher concurrency than the many-to-one model
- Allows another thread to execute when one thread makes a blocking system call
- Supports parallel execution on multiprocessor systems
- Drawback: creating many kernel threads can degrade system performance.
- Examples
 - Windows
 - Linux

Many-to-Many Model:



- The many-to-many model multiplexes many user-level threads to a smaller or equal number of kernel threads.
- The number of kernel threads may be specific to either a particular application or a particular machine.

Two-level Model:



- Similar to M:M, except that it allows a user thread to be **bound** to kernel thread

3. Thread libraries:

A **thread library** provides an API to create and manage threads. Thread libraries can be implemented in two ways:

1. **User-level thread library:**
Implemented entirely in user space without kernel support. Thread operations are simple function calls, not system calls.
2. **Kernel-level thread library:**
Supported directly by the operating system. Thread operations involve system calls to the kernel.

Commonly used thread libraries include **POSIX Pthreads**, **Windows threads**, and **Java threads**. Pthreads can be implemented at either user or kernel level. Windows threads are kernel-level, while Java threads are managed through the JVM and typically rely on the host OS thread library (Windows API or Pthreads).

In POSIX and Windows systems, **global variables are shared among all threads** of a process. Java does not support global variables, so shared data must be explicitly managed.

Thread creation can follow two strategies:

- **Asynchronous threading:**
The parent continues execution after creating a child thread. Threads run independently with minimal data sharing. Commonly used in servers and user interfaces.
- **Synchronous threading:**
The parent waits until all child threads finish execution before continuing. Threads often share data, and results may be combined by the parent.

Pthreads:

Pthreads (POSIX Threads) is a POSIX standard (IEEE 1003.1c) that defines an API for thread creation and synchronization. It is only a specification, not an actual implementation. Most UNIX-based operating systems such as Linux and macOS support Pthreads. Windows does not support it natively, but third-party implementations exist.

In a Pthreads program, threads begin execution in a specified function. The program includes the `pthread.h` header file. A thread identifier is declared using `pthread_t`, and thread attributes (such as stack size and scheduling) are defined using `pthread_attr_t`. If no attributes are explicitly set, default attributes are used.

A new thread is created using `pthread_create()`, where the function to be executed by the thread (for example, `runner()`) and any required arguments are passed. After creation, the program has two threads:

the parent thread running `main()`

the child thread executing the specified function

Both threads can share global data.

The parent thread waits for the child thread to finish using `pthread_join()`. The child thread terminates by calling `pthread_exit()`. After the child finishes execution, the parent can safely access shared results.

To handle multiple threads, `pthread_join()` is typically placed inside a loop to wait for all threads to complete.

Example:

```

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */
int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */
    /* set the default attributes of the thread */
    pthread_attr_t attr;
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid, NULL);
    printf("sum = %d\n", sum);
}
/* The thread will execute in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;
    for (i = 1; i <= upper; i++)
        sum += i;
    pthread_exit(0);
}
#define NUM_THREADS 10
/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];
for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);

```

Windows Threads:

The **Windows thread library** is similar to Pthreads in creating and managing threads. Programs using the Windows API must include the `windows.h` header file. Threads share **global data** (for example, `sum`), which is typically declared using the `DWORD` data type.

A separate thread executes a specific function (such as `Summation()`), which receives parameters through a `void*` pointer (`LPVOID` in Windows). The thread computes the required task and updates the shared global data.

Threads are created using the `CreateThread()` function. This function allows attributes such as security settings, stack size, and initial execution state. If default values are used, the thread becomes immediately eligible for CPU scheduling.

After creating the thread, the **parent thread must wait** for it to complete before accessing shared results. This is done using the `WaitForSingleObject()` function, which blocks the parent until the child thread exits.

To wait for **multiple threads**, the `WaitForMultipleObjects()` function is used. It allows the parent thread to wait until all or any specified threads finish execution, with an optional timeout.

This function is passed four parameters:

1. The number of objects to wait for
2. A pointer to the array of objects
3. A flag indicating whether all objects have been signaled
4. A timeout duration (or INFINITE)

Example:

```
#include <windows.h>
#include <stdio.h>
DWORD Sum; /* data is shared by the thread(s) */
/* The thread will execute in this function */
DWORD WINAPI Summation(LPVOID Param)
{
    DWORD Upper = *(DWORD*)Param;
    for (DWORD i = 1; i <= Upper; i++)
        Sum += i;
    return 0;
}
int main(int argc, char *argv[])
{
    DWORD ThreadId;
    HANDLE ThreadHandle;
    int Param;
    Param = atoi(argv[1]);
    /* create the thread */
    ThreadHandle = CreateThread(
        NULL, /* default security attributes */
        0, /* default stack size */
        Summation, /* thread function */
        &Param, /* parameter to thread function */
        0, /* default creation flags */
        &ThreadId); /* returns the thread identifier */
    /* now wait for the thread to finish */
    WaitForSingleObject(ThreadHandle, INFINITE);
    /* close the thread handle */
    CloseHandle(ThreadHandle);
    printf("sum = %d\n", Sum);
}
```

Java Threads:

in Java, **threads are the basic unit of program execution**. Every Java program has at least one thread; even a simple program with only a `main()` method runs as a single thread inside the JVM. Java threads are supported on all platforms that provide a JVM, such as Windows, Linux, macOS, and Android.

There are **two ways to create threads in Java**:

1. **Extending the `Thread` class** and overriding its `run()` method.
2. **Implementing the `Runnable` interface** (more commonly used), which defines a single method `run()`. The code inside `run()` executes in a separate thread.

To create a thread, a `Thread` object is created by passing a `Runnable` object, and the thread is started using the `start()` method. Calling `start()`:

- Creates and initializes a new thread in the JVM
- Internally invokes the `run()` method and makes the thread eligible for execution

(The `run()` method is never called directly.)

To make the **parent thread wait** until a child thread finishes, Java provides the `join()` method, which is similar to `pthread_join()` in Pthreads and `WaitForSingleObject()` in Windows. When multiple threads must be waited for, `join()` is typically used inside a loop.

```
class Task implements Runnable
{
    public void run()
    {
        System.out.println("I am a thread.");
    }
}
```

Thread creation in Java involves creating a `Thread` object and passing it an instance of a class that implements `Runnable`, followed by invoking the `start()` method on the `Thread` object.

This appears in the following example:

```
Thread worker = new Thread(new Task());
worker.start();
```

Invoking the `start()` method for the new `Thread` object does two things:

1. It allocates memory and initializes a new thread in the JVM.
2. It calls the `run()` method, making the thread eligible to be run by the JVM.

(Note again that we never call the `run()` method directly. Rather, we call the `start()` method, and it calls the `run()` method on our behalf.)

Recall that the parent threads in the Pthreads and Windows libraries use `pthread join()` and `WaitForSingleObject()` (respectively) to wait for the summation threads to finish before proceeding. The `join()` method in Java provides similar functionality. (Notice that `join()` can throw an `InterruptedException`, which we choose to ignore.)

```
try {
    worker.join();
} catch (InterruptedException ie) { }
```

4. Threading issues:

- Some of the issues to consider in designing multithreaded programs.
 1. The `fork()` and `exec()` System Calls.
 2. Signal Handling
 3. Thread Cancellation
 4. Thread-Local Storage
 5. Scheduler Activations

fork() and exec():

- The `fork()` system call is used to create separate, duplicate process.
- Does `fork()` duplicate only the calling thread or all threads?
 - Some UNIX OS's have two versions of `fork`.
- `exec()` usually works as normal – replace the running process including all threads.
- If `exec()` called immediately after forking, duplicating only the calling thread is appropriate.
- If, however, the separate process does not call `exec()` after forking, the separate process should duplicate all the threads.

Signal Handling:

- **Signals** are used in UNIX systems to notify a process that a particular event has occurred.

- A signal may be received may be synchronously or asynchronously depending on the source and the reason for an event.
- All signals, whether synchronously or asynchronously follow the same pattern.
 1. Signal is generated by particular event
 2. Signal is delivered to a process
 3. Once delivered, the Signal must be handled
- A signal may be handled by one of the two possible handlers:
 1. default
 2. user-defined
- Every signal has **default handler** that kernel runs when handling that signal
- **User-defined signal handler** that is called to handle the signal.
- Some signals(changing the size of window) are simply ignored.
- Others (illegal memory access) are handled by terminating a program.
- Handling signals in single-threaded program is straight forward;signals are always delivered to process.
- However, delivering signals is more complicated in multithreaded programs, where a process may have several threads
 1. Deliver the signal to the thread to which the signal applies
 2. Deliver the signal to every thread in the process
 3. Deliver the signal to certain threads in the process
 4. Assign a specific thread to receive all signals for the process

Thread Cancellation:

- Involves Terminating a thread before it has finished.
- Ex: webpage.
- Thread to be canceled is often referred to as **target thread**
- Two general approaches:
 - **Asynchronous cancellation** terminates the target thread immediately.
 - **Deferred cancellation** allows the target thread to periodically check if it should be cancelled.
- Invoking thread cancellation requests cancellation, but actual cancellation depends on thread state.

Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

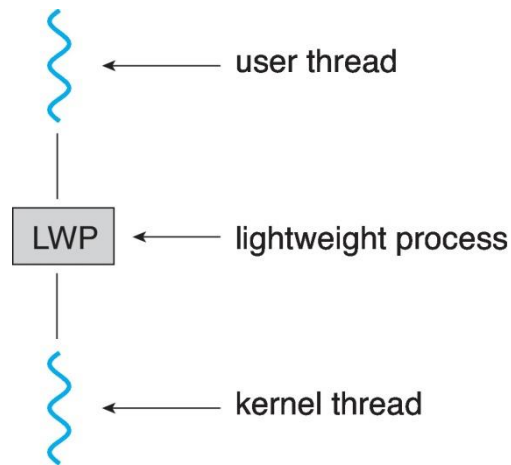
- If thread has cancellation disabled, cancellation remains pending until thread enables it
- Default type is deferred
 - Cancellation only occurs when thread reaches **cancellation point**

In Linux systems, thread cancellation is handled through signals.

Thread-Local Storage:

- Threads belongs to a process share the data of the process, this is the benefit of multi-threading.
- However, in some circumstances, each thread might need its own copy of certain data.
- We will call such data Thread-local storage (TLS)
Ex: transaction
- To associate each thread with its unique identifier, we could use Thread-local storage .

Scheduler Activations:



- Scheduler activation is a coordination technique between the kernel and a user-level thread library. The kernel provides the application with virtual processors (LWPs), and the thread library schedules user threads on them.
- When important events occur, the kernel notifies the thread library using an upcall (handled by an upcall handler running on a virtual processor).
 - If a thread is about to block:
The kernel sends an upcall identifying the thread and allocates a new virtual processor. The upcall handler saves the blocked thread's state, releases its processor, and schedules another ready thread.
 - When the blocked thread becomes ready:
The kernel sends another upcall. The thread library marks the thread as runnable and schedules it on an available virtual processor (allocating or preempting one if needed).

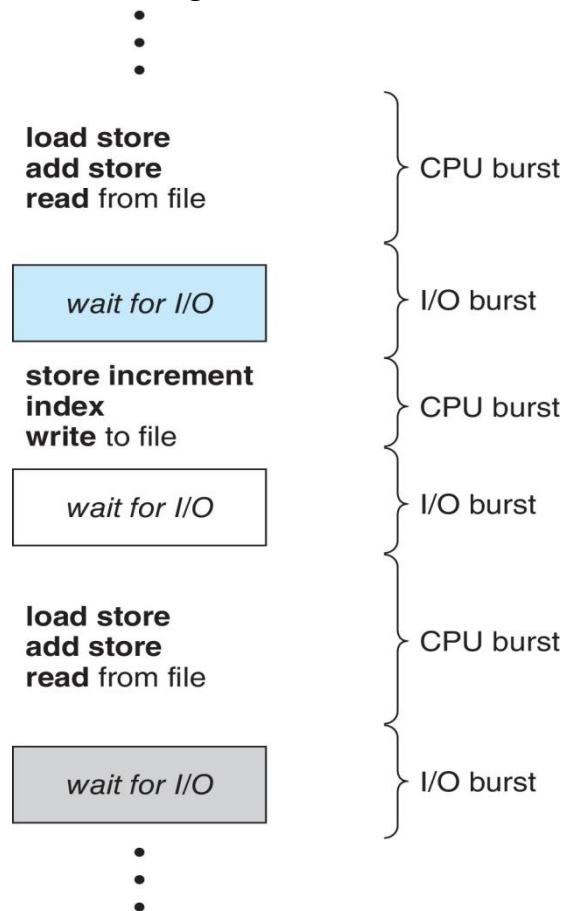
User-Level Thread (ULT)	Kernel-Level Thread (KLT)
Implemented by user-level libraries	Implemented by the Operating System
Not recognized by the OS	Recognized by the OS
Fast context switching with less overhead	Slower context switching with more overhead
Blocking blocks the entire process	Only the blocked thread is affected
Limited use of multiprocessing	Fully utilizes multiprocessing
Fast and simple creation and management	Slower and more complex management
Threads share the same address space	Each thread has its own address space
More portable, works on any OS	OS-dependent and less portable

Part-3

CPU Scheduling:

1.Basic concepts:

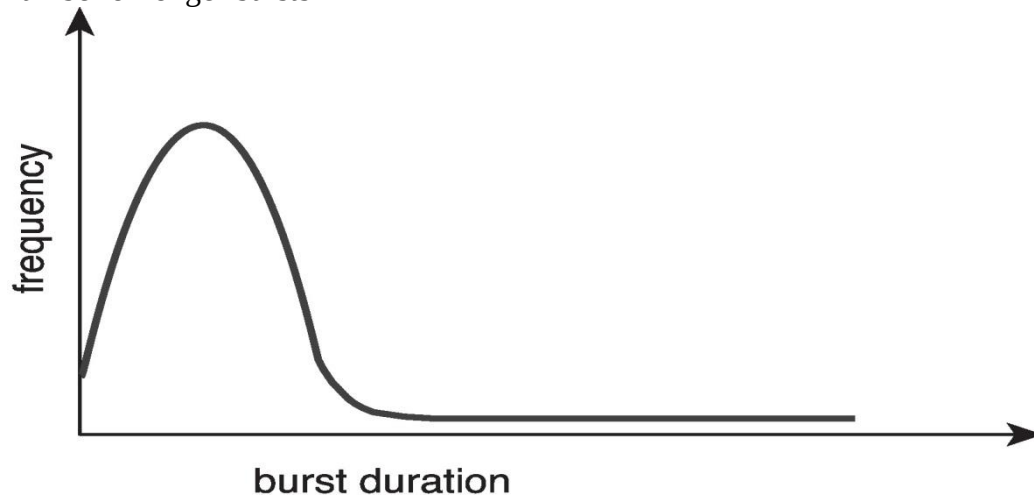
- Maximum CPU utilization obtained with multiprogramming
- Process execution consists of a **cycle** of CPU execution and I/O wait. Processes alternate between these two states.
- Process Execution begins with a CPU burst followed by an I/O burst which is followed by another CPU burst, then another I/O burst and so on final CPU burst ends with a system request to terminate execution.
- CPU burst is when the process is being executed in the CPU.
- I/O burst is when the CPU is waiting for I/O for further execution.



Histogram of CPU-burst Times

Large number of short bursts

Small number of longer bursts



CPU Scheduler

- Whenever the CPU becomes idle, the OS must select one of the processes in the ready queue to be executed. The selection process is carried out by the **CPU scheduler**. The **scheduler** selects the processes in the memory that are ready to execute and allocates the CPU to that process.
 - CPU scheduling decisions may take place when a process:
 - Switches from running to waiting state
 - Switches from running to ready state
 - Switches from waiting to ready
 - Terminates
 - For situations 1 and 4, there is no choice in terms of scheduling. A new process (if one exists in the ready queue) must be selected for execution.
 - For situations 2 and 3, however, there is a choice.

Preemptive and Nonpreemptive Scheduling

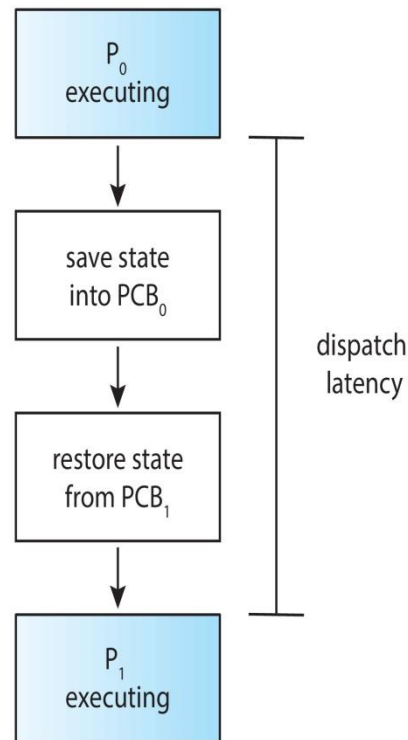
- When scheduling takes place only under circumstances 1 and 4, the scheduling scheme is **nonpreemptive** or cooperative.
- Otherwise, it is **preemptive**.
- Under Nonpreemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases it either by terminating or by switching to the waiting state.
- Virtually all modern operating systems including Windows, MacOS, Linux, and UNIX use preemptive scheduling algorithms.

Preemptive Scheduling	Non-Preemptive Scheduling
1.The CPU scheduling in which the resources (CPU Cycle) have been allocated to a process for a limited amount of time is known as preemptive scheduling.	The type of scheduling in which once the resources (CPU Cycle) have been allocated to a process, the process holds it until it completes its burst time or switches to the 'wait' state is known as non-preemptive scheduling.
2.In the preemptive scheduling, a process can be interrupted when it is being executed.	In the non-preemptive scheduling, a process cannot be interrupted until it terminates itself or its time is over.
3.If a process that has a high priority arrives frequently in the 'ready' queue, the low priority processes may starve.	If a process that has a long burst time is running the CPU, then the process that has less CPU burst time would starve
4.It has overheads.	It does not have overheads.
5.It is flexible in nature.	It is not flexible in nature.
6.It is expensive.	It is not expensive.
7.Examples ? Round Robin scheduling, Shortest Remaining Time First scheduling.	Examples ? First Come First Serve and Shortest Job First.

Dispatcher

- Dispatcher is the module gives control of the CPU to the process selected by the CPU scheduler; this involves:

- Switching context
 - Switching to user mode
 - Jumping to the proper location in the user program to restart that program
- **Dispatch latency** – the time it takes for the dispatcher to stop one process and start another running.



2.Scheduling criteria:

CPU scheduling is essential for the system's performance and ensures that processes are executed correctly and on time. Different CPU scheduling algorithms have other properties and the choice of a particular algorithm depends on various factors. Many criteria have been suggested for comparing CPU scheduling algorithms.

Criteria of CPU Scheduling

Now let's discuss CPU Scheduling has several criteria. Some of them are mentioned below.

1. CPU utilization: The main objective of any CPU scheduling algorithm is to keep the CPU as busy as possible. Theoretically, CPU utilization can range from 0 to 100 but in a real-time system, it varies from 40 to 90 percent depending on the load upon the system.

2. Throughput: A measure of the work done by the CPU is the number of processes being executed and completed per unit of time. This is called throughput. The throughput may vary depending on the length or duration of the processes.

3. Turnaround Time: For a particular process, an important criterion is how long it takes to execute that process. The time elapsed from the time of submission of a process to the time of completion is known as the turnaround time. Turn-around time is the sum of times spent waiting to get into memory, waiting in the ready queue, executing in CPU and waiting for I/O.

Turn Around Time = Completion Time - Arrival Time.

4. **Waiting Time:** A scheduling algorithm does not affect the time required to complete the process once it starts execution. It only affects the waiting time of a process i.e. time spent by a process waiting in the ready queue.

Waiting Time = Turnaround Time - Burst Time.

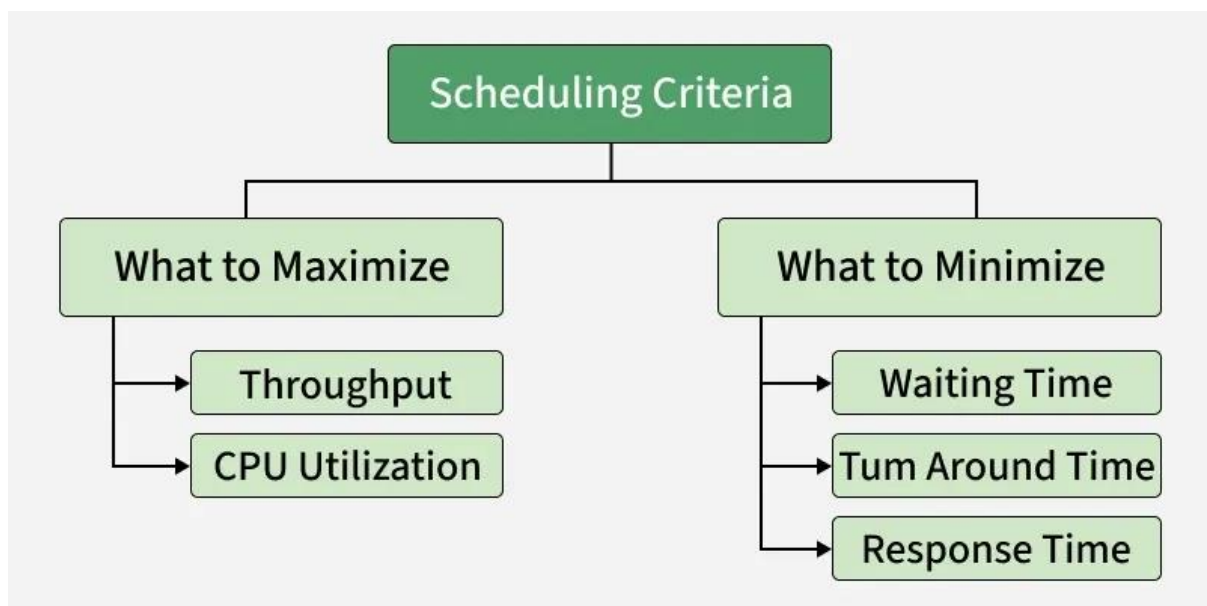
5. **Response Time:** In an interactive system, turn-around time is not the best criterion. A process may produce some output fairly early and continue computing new results while previous results are being output to the user. Thus another criterion is the time taken from submission of the process of the request until the first response is produced. This measure is called response time.

Response Time = CPU Allocation Time (when the CPU was allocated for the first) - Arrival Time

Terminologies Used in CPU Scheduling

- **Arrival Time:** The time at which the process arrives in the ready queue.
- **Completion Time:** The time at which the process completes its execution.
- **Burst Time:** Time required by a process for CPU execution.
- **Turn Around Time:** Time Difference between completion time and arrival time.
- **Waiting Time(W.T):** Time Difference between turn around time and burst time.

Scheduling Algorithm Optimization Criteria



3. Scheduling algorithms:

CPU Scheduling Algorithms

CPU scheduling is a process used by the operating system to decide which task or process gets to use the CPU at a particular time. This is important because a CPU can only handle one task at a time, but there are usually many tasks that need to be processed. The following are different purposes of a CPU scheduling time.

- Maximize the CPU utilization
- Minimize the response and waiting time of the process.

Different Types of CPU Scheduling Algorithms

There are mainly two types of scheduling methods:

Preemptive Scheduling: Preemptive scheduling is used when a process switches from running state to ready state or from the waiting state to the ready state.

Non-Preemptive Scheduling: Non-Preemptive scheduling is used when a process terminates , or when a process switches from running state to waiting state.



a).First- Come, First-Served (FCFS) Scheduling

- Jobs are executed on first come, first serve basis.
- It is a non-preemptive scheduling algorithm.
- Easy to understand and implement.
- Its implementation is based on FIFO queue.
- Poor in performance as average wait time is high.

Example1:

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3

The Gantt Chart for the schedule is:



Completion Time: $P_1 = 24$, $P_2 = 27$, $P_3 = 30$

Turnaround Time(TAT) = Completion Time(CT) – Arrival Time(AT)

$$P_1=24-0=24, P_2=27-0=27, P_3=30-0=30$$

Average Turnaround Time(ATAT) = Total turn around time of all processes / Total no of processes

$$=(24+27+30)/3=27$$

Waiting time = Turnaround Time(TAT) –Burst Time(BT)

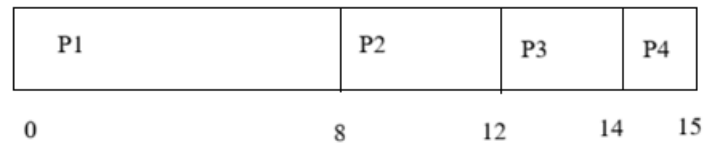
$$P_1 = 0-0=0; P_2 = 27-3=24; P_3 = 30-3=27$$

Average waiting time: $(0 + 24 + 27)/3 = 17$

Example2:

<u>Process</u>	<u>Burst Time</u>	<u>Arrival Time</u>
P_1	8	0
P_2	4	2
P_3	2	4
P_4	1	6

The Gantt Chart for the schedule is:



Completion Time:

$$P1=8, P2=12, P3=14, P4=15$$

Turnaround Time(TAT):

$$\text{Turnaround Time(TAT)} = \text{Completion Time(CT)} - \text{Arrival Time(AT)}$$

$$P1=8-0=8$$

$$P2=12-2=10$$

$$P3=14-4=10$$

$$P4=15-6=9$$

$$\text{Average Turnaround Time(ATAT)} = \frac{\text{Total turn around time of all processes}}{\text{Total no of processes}}$$

$$\text{Average Turnaround Time(ATAT)} = (8+10+10+9)/4 = 37/4 = 9.25\text{ms}$$

Waiting Time: $\text{Waiting Time} = \text{Turnaround Time(TAT)} - \text{Burst Time(BT)}$

$$P1=8-8=0, P2=10-4=6, P3=10-2=8, P4=9-1=8$$

$$\text{Average Waiting Time} = (0+6+8+8)/4 = 22/4 = 5.5\text{ms}$$

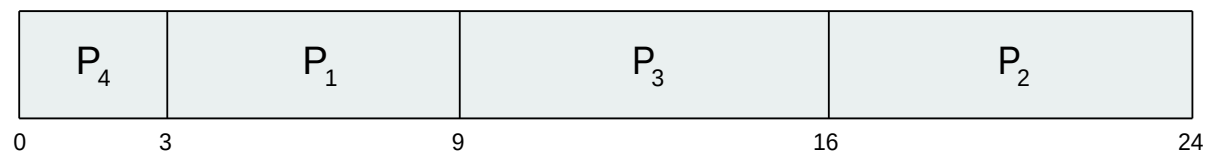
b.Shortest-Job-First (SJF) Scheduling:

- This is also known as **shortest job first**, or SJF
- This is a non-preemptive scheduling algorithm.
- Best approach to minimize waiting time.
- Easy to implement in Batch systems where required CPU time is known in advance.
- Impossible to implement in interactive systems where required CPU time is not known.
- The processor should know in advance how much time process will take.

Example1:

Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

SJF scheduling chart



Completion Time: $P1 = 9, P2 = 24, P3 = 16, P4 = 3$

Turnaround Time(TAT) = Completion Time(CT) – Arrival Time(AT)

$$P1=9, P2=24, P3=16, P4=3$$

$$\text{Average Turnaround Time(ATAT)} = \frac{\text{Total turn around time of all processes}}{\text{Total no of processes}}$$

$$= (9+24+16+3)/4 = 52/4 = 13.00$$

Waiting time = Turnaround Time(TAT) –Burst Time(BT)

$$P1 = 9-6=3; P2 = 24-8=16; P3 = 16-7=9, P4=3-3=0$$

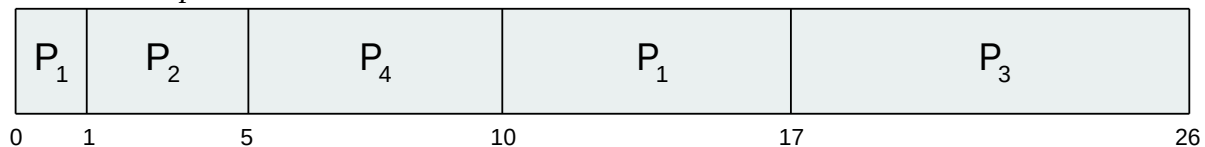
Average waiting time: $(3+16+9+0)/4 = 7$

c. Shortest Remaining Time First Scheduling

- The pre-emptive version of Shortest Job First (SJF) scheduling is called Shortest Remaining Time First (SRTF).
- Whenever a new process arrives in the ready queue, the decision on which process to schedule next is redone using the SJN algorithm.
- the SRTF algorithm involves more overheads than the Shortest job first (SJF) scheduling, because in SRTF OS is required frequently in order to monitor the CPU time of the jobs in the **READY** queue and to perform context switching.
- At any moment, the CPU checks all **arrived processes** and picks the one that has the **smallest remaining execution time**.

Process	Arrival Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

- *Preemptive SJF Gantt Chart*



Completion Time: $P_1 = 17$, $P_2 = 5$, $P_3 = 26$, $P_4 = 10$

Turnaround Time(TAT) = Completion Time(CT) – Arrival Time(AT)

$P_1 = 17 - 0 = 17$, $P_2 = 5 - 1 = 4$, $P_3 = 26 - 2 = 24$, $P_4 = 10 - 3 = 7$

Average Turnaround Time(ATAT) = Total turn around time of all processes / Total no of processes

$= (17 + 4 + 24 + 7) / 4 = 52 / 4 = 13.00$

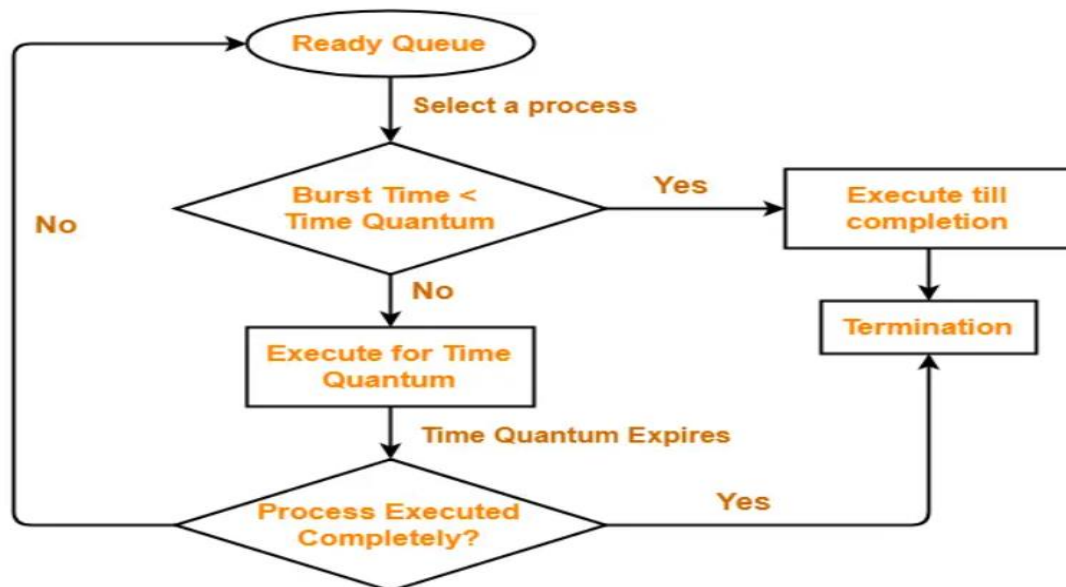
Waiting time = Turnaround Time(TAT) – Burst Time(BT)

$P_1 = 17 - 8 = 9$; $P_2 = 4 - 4 = 0$; $P_3 = 24 - 9 = 15$, $P_4 = 7 - 5 = 2$

Average waiting time: $(9 + 0 + 15 + 2) / 4 = 26 / 4 = 6.5$

d. Round Robin (RR):

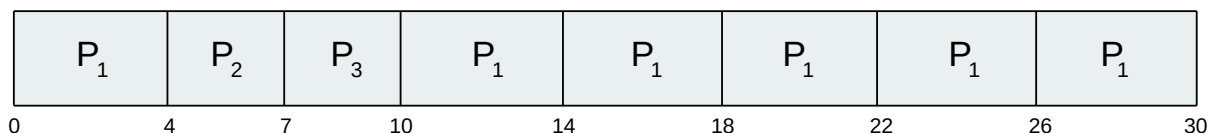
- Round Robin(RR) scheduling algorithm is mainly designed for time-sharing systems.
- The **round-robin (RR)** scheduling algorithm is similar to FCFS scheduling, but preemption is added to enable the system to switch between processes.
- A small unit of time, called a **time quantum** or **time slice**, is defined. A time quantum is generally from 10 to 100 milliseconds in length.
- The ready queue is treated as a circular queue. The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.
- **New processes** are added at the **tail** of the ready queue.
- Scheduler picks the **first process**, sets a **timer** for 1 time quantum, and dispatches it.
- **Case 1:** If process CPU burst < **time quantum**, it **releases CPU voluntarily**, and scheduler picks the next process.
- **Case 2:** If CPU burst > **time quantum**, timer interrupt occurs → **context switch** → process moves to **tail of queue**.
- Scheduler then chooses the **next process** in the queue.



Round Robin Scheduling

Process	Burst Time
P_1	24
P_2	3
P_3	3

The Gantt chart is:



Completion Time: $P_1 = 30$, $P_2 = 7$, $P_3 = 10$

Turnaround Time(TAT) = Completion Time(CT) – Arrival Time(AT)

$P_1=30-0=30, P_2=7-0=7, P_3=10-0=10$

Average Turnaround Time(ATAT) = Total turn around time of all processes / Total no of processes

$= (30+7+10)/3 = 47/3 = 15.66$

Waiting time = Turnaround Time(TAT) –Burst Time(BT)

$P_1 = 30-24=6, P_2 = 7-3=4; P_3 = 10-3=7$

Average waiting time: $(6+4+7)/3 = 17/3 = 5.66$

e.Priority Scheduling:

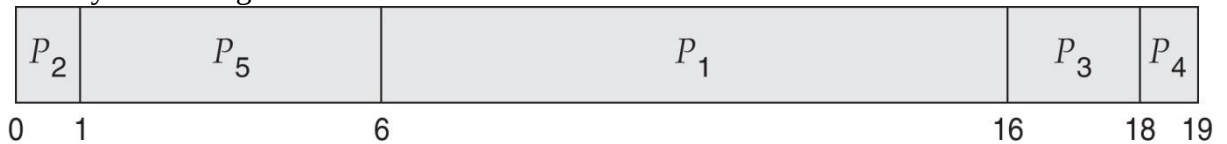
- The SJF algorithm is a special case of the general priority-scheduling algorithm.
- A priority is associated with each process, and the CPU is allocated to the process with the highest priority.
- Equal-priority processes are scheduled in FCFS order.
- Priorities are generally indicated by some fixed range of numbers, such as 0 to 7 or 0 to 4,095.
- However, there is no general agreement on whether 0 is the highest or lowest priority.
- Some systems use low numbers to represent low priority; others use low numbers for high priority.
- priority scheduling can be either preemptive or nonpreemptive.

Example:

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4

P_4 1 5
 P_5 5 2

Priority scheduling Gantt Chart



Completion Time: $P_1 = 16$, $P_2 = 1$, $P_3 = 18$, $P_4 = 19$, $P_5 = 6$

Turnaround Time(TAT) = Completion Time(CT) – Arrival Time(AT)

$P_1 = 16$, $P_2 = 1$, $P_3 = 18$, $P_4 = 19$, $P_5 = 6$

Average Turnaround Time(ATAT) = Total turn around time of all processes / Total no of processes

$= (16 + 1 + 18 + 19 + 6) / 5 = 65 / 5 = 13.00$

Waiting time = Turnaround Time(TAT) – Burst Time(BT)

$P_1 = 16 - 10 = 6$, $P_2 = 1 - 1 = 0$, $P_3 = 18 - 2 = 16$, $P_4 = 19 - 1 = 18$, $P_5 = 6 - 5 = 1$

Average waiting time: $(6 + 0 + 16 + 18 + 1) / 5 = 41 / 5 = 8.2$

f.Multilevel Queue:

- The ready queue consists of multiple queues
- Multilevel queue scheduler defined by the following parameters:
 - Number of queues
 - Scheduling algorithms for each queue
 - Method used to determine which queue a process will enter when that process needs service
 - Scheduling among the queues.
- With priority scheduling, have separate queues for each priority.
- Schedule the process in the highest-priority queue

priority = 0

T_0	T_1	T_2	T_3	T_4
-------	-------	-------	-------	-------

priority = 1

T_5	T_6	T_7
-------	-------	-------

priority = 2

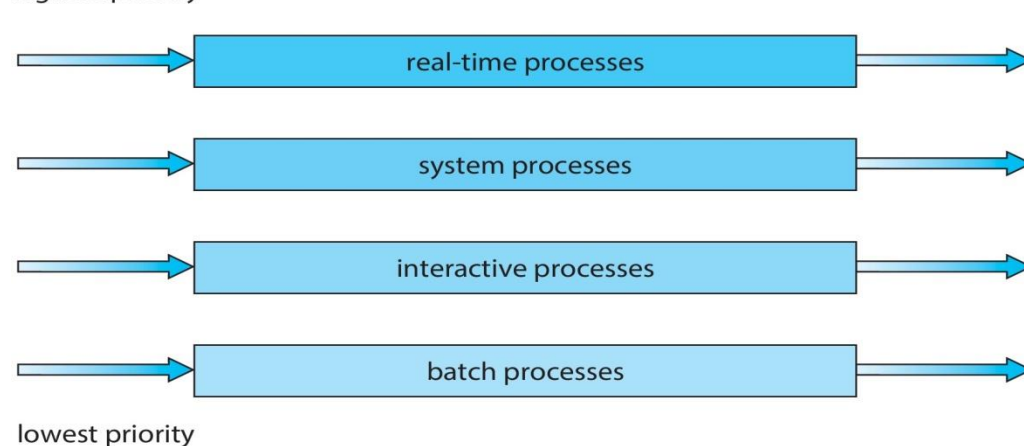
T_8	T_9	T_{10}	T_{11}
-------	-------	----------	----------

•
•
•

priority = n

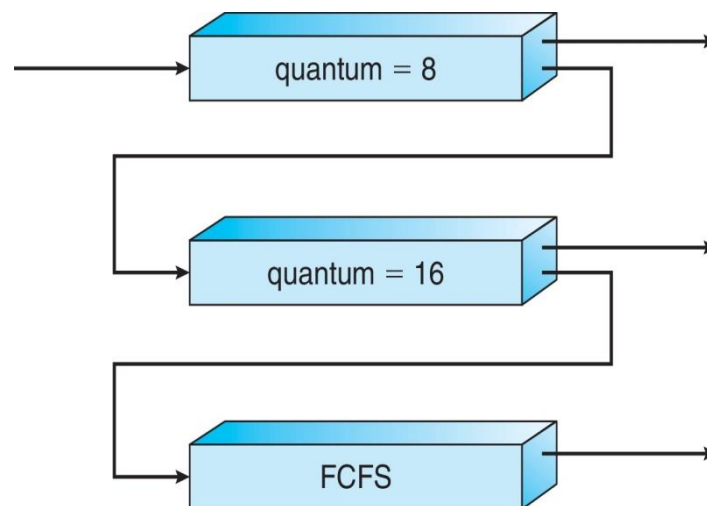
T_x	T_y	T_z
-------	-------	-------

- Prioritization based upon process type



g. Multilevel Feedback Queue:

- A process can move between the various queues.
- Multilevel-feedback-queue scheduler defined by the following parameters:
 - Number of queues
 - Scheduling algorithms for each queue
 - Method used to determine when to upgrade a process
 - Method used to determine when to demote a process
 - Method used to determine which queue a process will enter when that process needs service
- Aging can be implemented using multilevel feedback queue
- Three queues:
 - Q_0 – RR with time quantum 8 milliseconds
 - Q_1 – RR time quantum 16 milliseconds
 - Q_2 – FCFS
- Scheduling
 - A new process enters queue Q_0 which is served in RR
 - When it gains CPU, the process receives 8 milliseconds
 - If it does not finish in 8 milliseconds, the process is moved to queue Q_1
 - At Q_1 job is again served in RR and receives 16 additional milliseconds
 - If it still does not complete, it is preempted and moved to queue Q_2



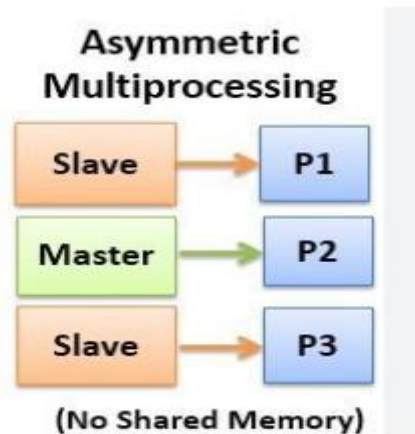
4. Multiple processor scheduling:

- CPU scheduling more complex when multiple CPUs are available
- Traditionally, the term **multiprocessor** referred to systems that provided multiple physical processors, where each processor contained one single-core CPU.
- In modern computing systems, **multiprocessor** now applies to the following system architectures:
 - Multicore CPUs
 - Multithreaded cores
 - NUMA systems
 - Heterogeneous multiprocessing

Asymmetric Multiprocessing (AMP)

- **One master processor** handles:
 - All scheduling decisions
 - I/O and system activities
- Other processors run **only user code**

- Advantages: simple, no complex synchronization
- Disadvantages: master becomes **bottleneck**



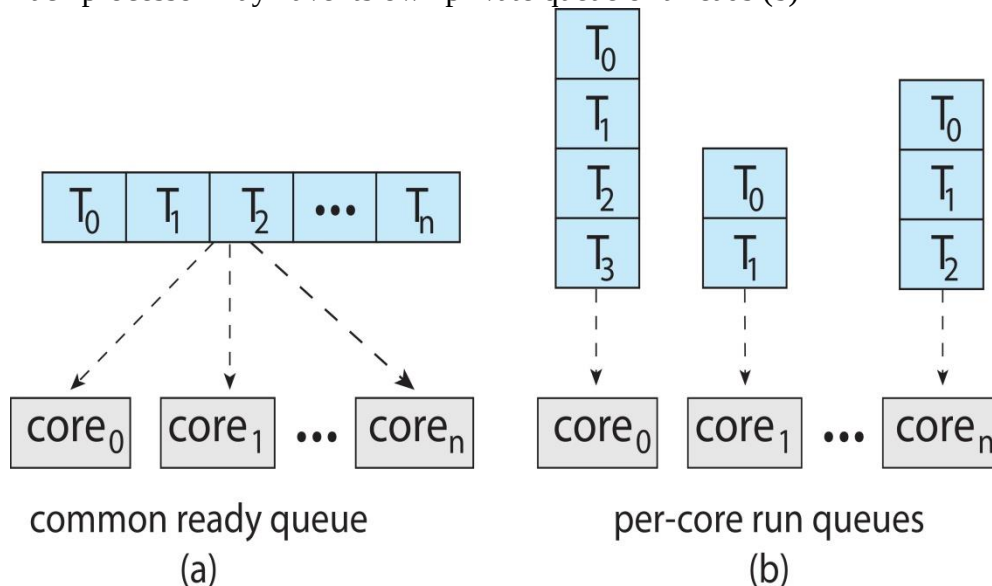
Symmetric multiprocessing (SMP)

Symmetric multiprocessing (SMP) is where each processor is self scheduling.

Two modes of task ques.

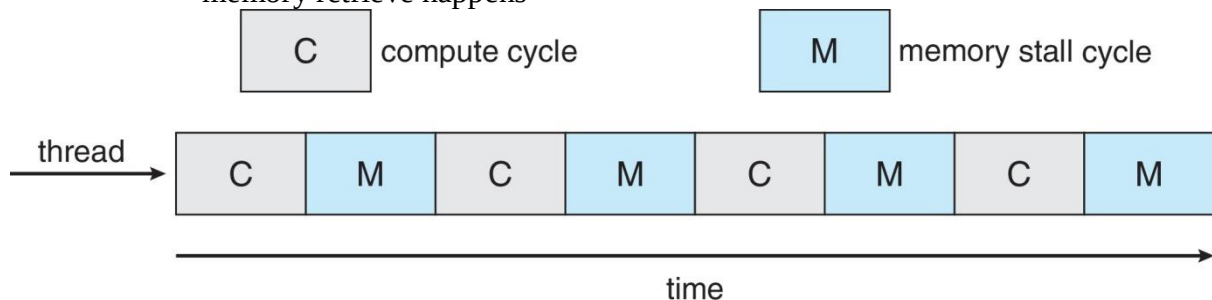
All threads may be in a common ready queue (a)

Each processor may have its own private queue of threads (b)



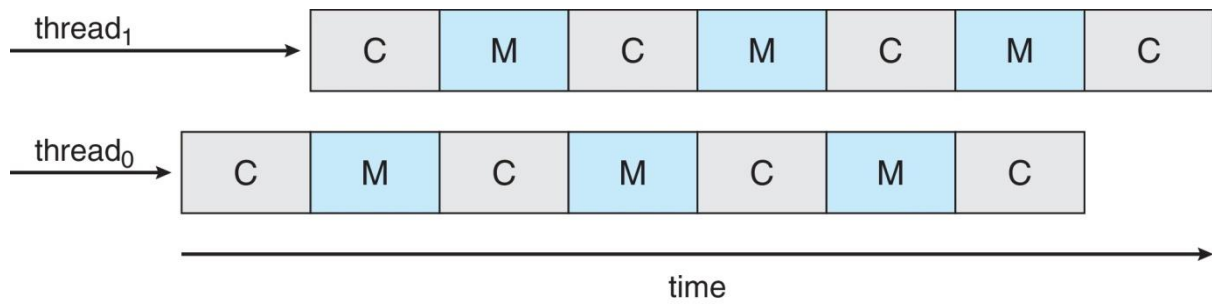
Multicore Processors

- Recent trend to place multiple processor cores on same physical chip
- Faster and consumes less power
- Multiple threads per core also growing
 - Takes advantage of memory stall to make progress on another thread while memory retrieve happens

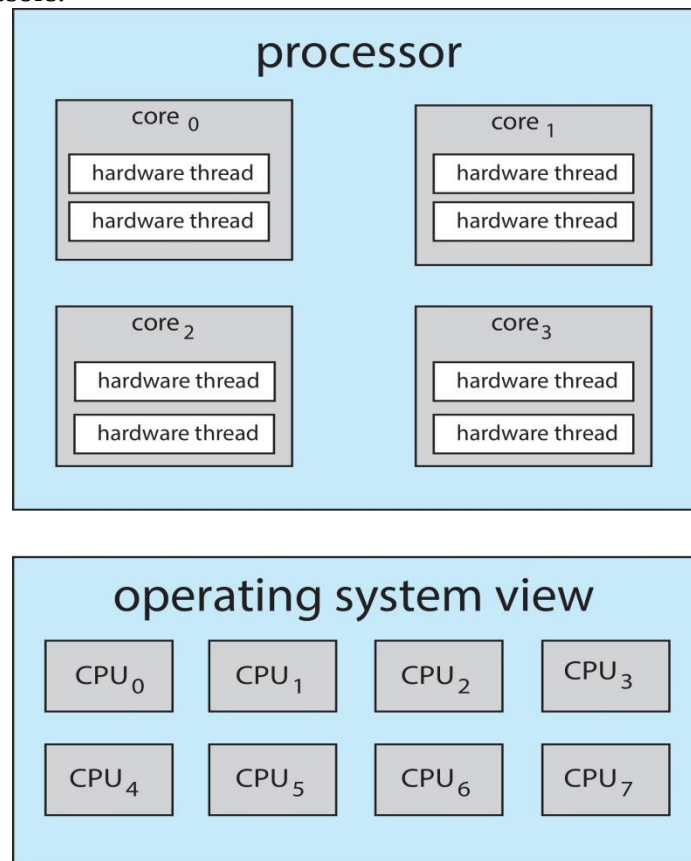


Multithreaded Multicore System

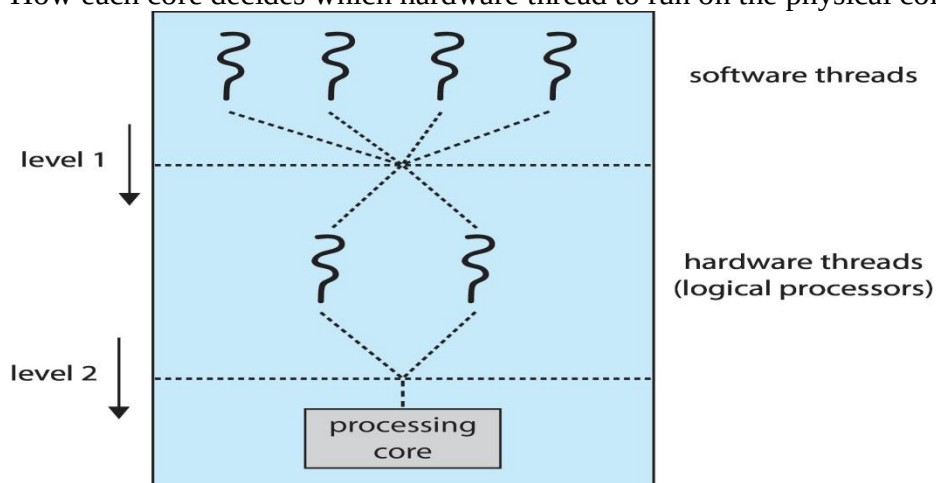
- Each core has > 1 hardware threads.
- If one thread has a memory stall, switch to another thread



- **Chip-multithreading (CMT)** assigns each core multiple hardware threads. (Intel refers to this as **hyperthreading**.)
- On a quad-core system with 2 hardware threads per core, the operating system sees 8 logical processors.



- Two levels of scheduling:
 1. The operating system deciding which software thread to run on a logical CPU
 2. How each core decides which hardware thread to run on the physical core.



Multiple-Processor Scheduling – Load Balancing

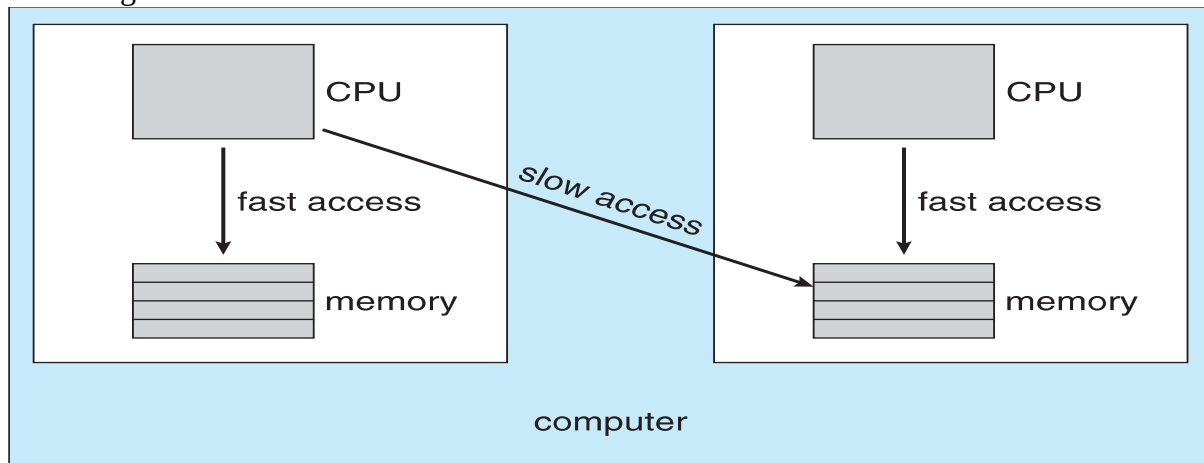
- If SMP, need to keep all CPUs loaded for efficiency
- **Load balancing** attempts to keep workload evenly distributed
- **Push migration** – periodic task checks load on each processor, and if found pushes task from overloaded CPU to other CPUs
- **Pull migration** – idle processors pulls waiting task from busy processor

Multiple-Processor Scheduling – Processor Affinity

- When a thread has been running on one processor, the cache contents of that processor stores the memory accesses by that thread.
- We refer to this as a thread having affinity for a processor (i.e., “processor affinity”)
- Load balancing may affect processor affinity as a thread may be moved from one processor to another to balance loads, yet that thread loses the contents of what it had in the cache of the processor it was moved off of.
- **Soft affinity** – the operating system attempts to keep a thread running on the same processor, but no guarantees.
- **Hard affinity** – allows a process to specify a set of processors it may run on.

NUMA(Non-Uniform Memory Access) and CPU Scheduling

If the operating system is **NUMA-aware**, it will assign memory close to the CPU the thread is running on.



NUMA(Non-Uniform Memory Access):

- Each CPU/core has **local memory** (fast access)
- Accessing remote memory is slow

Scheduler Requirements

- Keep thread close to memory it accesses
- Avoid remote-memory penalties

OS combines CPU scheduling + memory placement policies

Heterogeneous Multiprocessing:

- Heterogeneous multiprocessing refers to a CPU architecture in which multiple processors/cores exist, but they are not identical in terms of:
 - Performance
 - Power consumption
 - Instruction capabilities
 - Internal microarchitecture
- Unlike traditional SMP (Symmetric Multiprocessing), where all cores are identical, in HMP each processor type is designed for a specific class of tasks.

The scheduler must decide:

- Whether a thread should run on a **big** or **LITTLE** core
- When to migrate a thread (e.g., low load → LITTLE, burst load → big)
- Power and thermal constraints